Guía de construcción del laboratorio de reservas con Kubernetes

Montaje por etapas y contratos de trabajo para los seis grupos

Guía para construir y evaluar el laboratorio Kubernetes del sistema de reservas con carga progresiva y fallos controlados. Incluye dependencias, criterios de aceptación y rutas de los prompts para otro agente. El despliegue y la meta de 50 000 usuarios siguen pendientes de ejecución.

Índice

Actualizar el índice en Word con F9.

DOC-02-02 · 20 de octubre de 2026 · Plan de construcción para otro agente · Estado propuesto sin ejecución

Propósito y punto de partida

Construir un laboratorio reproducible para evaluar cómo la arquitectura Kubernetes mantiene el servicio de reservas cuando aumenta la demanda y falla un componente. La meta experimental es llegar a 50 000 usuarios virtuales activos en un perfil definido, además de estudiar una campaña de 50 000 intenciones de reserva. Esas dos pruebas tendrán resultados separados. La investigación no presupone que se alcanzará la meta ni que toda interrupción será invisible.

Ya existen frontend, API FastAPI, PostgreSQL, outbox, pruebas Compose y manifiestos iniciales. La entrega documental anterior quedó en el commit 56927b4; no contiene un laboratorio distribuido validado. S19 sigue como trabajo local en revisión y sus 3240 HTTP medidos no acreditan 50 000 usuarios ni admisión Kubernetes. Esta guía autoriza preparar el alcance documental; no constituye autorización para contratar nube, alterar servicios compartidos o ejecutar fallos sobre infraestructura ajena.

Leer primero AGENTS.md, ESTADO-ACTUAL-SEMINARIO.md, DOC-01-02 y los originales de organización y caso de éxito de Fase 01. Preservar el sistema de reservas y sus contratos. Cada etapa debe terminar con evidencia o un pendiente explícito; commit y push requieren aprobación de su entrega según las instrucciones del proyecto.

Arquitectura que se debe construir

El tráfico debe recorrer generadores externos, DNS y entrada HTTPS redundante, frontend con proxy, API replicada y el endpoint de escritura PostgreSQL. El plano de control administra el clúster; no procesa la reserva del participante. Mantener la base y la entrada disponibles es tan necesario como disponer de varias API. El diagrama editable está en diagramas/laboratorio-ha-50000/arquitectura.mmd.

Perfil	Construcción	Alcance de la conclusión
L local	kind sobre un equipo con varios nodos lógicos	Ensayo de configuración y mecanismos; comparte fallo físico
D distribuido	Clúster con nodos y almacenamiento en dominios de fallo identificados	Carga y continuidad ante los fallos efectivamente ensayados
O OpenShift	Perfil distribuido sobre OpenShift y sus operadores compatibles	Conclusiones específicas de OpenShift tras admisión y pruebas

En D usar como topología inicial tres nodos de trabajo y plano de control HA administrado o tres miembros de control con etcd y endpoint redundante. Repartir tres réplicas iniciales de frontend y

API entre nodos mediante restricciones de topología; conservar capacidad para perder un trabajador. El número tres es una hipótesis inicial de arquitectura, no un dimensionamiento suficiente para 50 000 usuarios. [R1, R2]

Proponer PostgreSQL con tres instancias, un primario y dos réplicas, gestionadas por un operador compatible o servicio administrado. Documentar endpoint de escritura, promoción automática, prevención de doble primario, reconexión, almacenamiento independiente y política de replicación. Un PVC o StatefulSet por sí solo no configura alta disponibilidad. La redundancia de entrada, DNS y almacenamiento debe tener pruebas propias; si algo queda único, limitar expresamente la conclusión. [R5, R8]

Etapa L00 de inventario y decisiones

El agente coordinador inspeccionará sin modificar el entorno. Registrará CPU, RAM asignada y libre, disco e IOPS, red, virtualización, contextos Kubernetes, servicios y puertos ocupados, proveedor disponible y permisos. No incluirá kubeconfigs, claves ni direcciones privadas sensibles en evidencias compartidas. laboratorio/infraestructura/inventario.md está pendiente de comprobación y debe contrastarse con el equipo real.

Entregar laboratorio/ha-50000/inventario.md, decisiones.md y versions.lock.yaml. Esos archivos todavía deben construirse. Incluir versiones y digest de imágenes, compatibilidad Kubernetes u OpenShift, CNI con políticas de red, CSI y dominios de fallo, entrada TLS, operador de PostgreSQL, herramientas de observación y generador. No reutilizar automáticamente versiones antiguas ni instalar latest sin fijarlo. Revisar los scripts existentes: New-Lab.ps1 exige versiones concretas y kind-32gb.yaml contiene un control y dos trabajadores, no la topología D.

El presupuesto se obtiene con un piloto: medir recursos por componente y por usuario virtual, conexiones, almacenamiento por reserva y crecimiento de telemetría. Para una tasa objetivo λ y duración media de una iteración W, estimar usuarios del generador con λ por W y añadir margen medido. Esa estimación sirve para tasa abierta; 50 000 VU activos exige además medir el costo de sostenerlos. No deducir RAM de un nombre de perfil ni capacidad de hardware de contadores de Docker.

La salida obligatoria de L00 es una ruta concreta con máquinas, proveedor, costos si existen, nombres de contexto y namespaces, supuestos pendientes y alcance de fallos. Preparar comandos específicos después de conocer esos datos. Detener solamente la provisión dependiente si falta acceso o presupuesto; continuar manifiestos, documentación y validaciones locales independientes. No ejecutar scripts sobre el contexto por defecto.

Etapas L01 y L02 de plataforma y aplicación

Construir en laboratorio/ha-50000 los directorios infra, plataforma, datos, app, observabilidad, scripts y runbooks. Usar overlays local y distribuido; agregar openshift solo si esa es la plataforma elegida. Mantener separación de namespaces para aplicación, observación y datos según los permisos del entorno. Los nombres y selectores finales se registran en decisiones.md.

G1 debe proporcionar provisión reproducible o instrucciones exactas para las máquinas disponibles, instalación del clúster, CNI, almacenamiento y entrada, verificación y recuperación.

Cada comando de escritura exige contexto y namespace explícitos. Los scripts deben admitir vista previa, fallar ante errores y conservar recursos existentes. Los comandos de destrucción quedan separados y requieren identificación inequívoca del laboratorio y autorización de ese alcance.

Después construir imágenes versionadas del frontend y backend, publicar o cargar en el registro autorizado y registrar digest. Adaptar aplicacion/kubernetes/base.yaml y frontend/deploy/base; no aplicar la base antigua junto con una segunda base PostgreSQL del operador. Separar inicialización y migración en un Job único antes del despliegue de API y worker. No inicializar esquema en cada réplica. Proveer configuración y secretos por canal privado y asegurar acceso desde todos los nodos al registro.

Configurar startup, readiness, liveness, tiempos de terminación y retirada de tráfico. La sonda de vida no debe reiniciar todas las API por una caída transitoria de PostgreSQL. Reutilizar /health/live y /health/ready con pruebas reales; el frontend tiene su propio recurso de salud. Mantener /api en el proxy hacia las rutas backend sin prefijo. Comprobar HTTPS, balanceo, DNS, endpoints, permisos y almacenamiento antes de generar carga.

El control inicial debe demostrar catálogo, disponibilidad, reserva 201, reenvío 200 con mismo ID, consulta autorizada y salida de sesión. Un envío con la misma clave y contenido distinto debe producir idempotency_conflict. Probar dos propietarios con aislamiento de consulta. Conservar evidencias del navegador y API separadas: los VU HTTP no son navegadores renderizando JavaScript.

Etapa L03 de datos e integridad

G4 construirá el clúster PostgreSQL y su runbook de promoción, recuperación, copia y restauración. Fijar RPO propuesto cero para reservas que recibieron confirmación y ensayarlo; documentar cómo la replicación síncrona seleccionada equilibra ese objetivo con disponibilidad ante pérdida de réplicas. La replicación asíncrona puede perder escrituras confirmadas durante una promoción; no prometer RPO cero sin evidencia y configuración apropiada. [R5]

La aplicación confirma después del commit de descuento, reserva y outbox. Conservar ese contrato: Kafka sería una ampliación posterior para eventos confirmados, no un reemplazo de la escritura final. En la primera comparación dejar el worker desactivado en todas las condiciones para aislar la API y crecimiento de outbox; validar el worker y mensajería en otra etapa con su propia referencia.

El código actual protege cupos mediante UPDATE condicional sobre una fila de events. Un solo evento muy demandado genera contención en esa fila; agregar API no elimina ese límite. Registrar espera por bloqueos, conexiones, transacciones y latencia de almacenamiento. Comparar evento único y múltiples eventos como experimentos distintos. No cambiar el modelo de inventario en mitad de una comparación para declarar que Kubernetes resolvió el bloqueo.

Cada proceso API crea su pool. Presupuestar conexiones máximas como réplicas máximas por procesos por POOL_MAX, más worker, migraciones, observación y reserva administrativa. Revisar POOL_MAX actual de diez antes de ampliar; un pooler es una intervención adicional y exige compatibilidad y medición. Dimensionar timeouts y reintentos acotados conservando la misma clave en resultados inciertos.

Generar datos nuevos por ensayo y guardar auditoría previa y posterior: capacity menos available debe coincidir con la suma de quantity de reservas confirmadas de cada evento; no debe haber available negativo, reservas pending persistidas, duplicados de client_id más idempotency_key ni outbox ausente o huérfano. Comparar IDs confirmados al cliente con los de la base después del fallo y resolver las operaciones inciertas con la clave original. No modificar la base medida de S19. La inicialización existente no reinicia eventos previos y admite hasta un millón de cupos; cada ensayo requiere una base exclusiva o una semilla controlada nueva.

Etapa L04 de observación y autoescalado

G5 instalará métricas de nodos, contenedores, objetos del clúster y PostgreSQL, Prometheus y Grafana con almacenamiento y retención presupuestados. Exportar series y eventos fuera del dominio de fallo ensayado o registrar las lagunas. Sin reloj sincronizado y señal externa no se puede medir con precisión el inicio y fin de una interrupción.

Agregar histogramas de latencia por ruta y resultado, sin claves ni UUID en etiquetas. Las métricas actuales contienen suma y conteo; permiten media pero no p95. Validar histogramas con observaciones del cliente, reinicios de contadores y agregación entre réplicas. Registrar tasa ofrecida y efectiva, 201 únicos, reenvíos 200, sold_out, conflictos, timeouts, 5xx, CPU, throttling, memoria, OOM, conexiones y bloqueos. Los percentiles no se promedian entre Pods ni corridas. [R7]

G2 preparará tres tratamientos: A una API fija, B tres API fijas y C HPA de tres a un máximo presupuestado por L00. Conservar frontend, datos, recursos por Pod y carga iguales; contabilizar el presupuesto diferente. Probar metrics.k8s.io y requests antes de habilitar HPA. El HPA existente de dos a seis réplicas y CPU al 60 por ciento es un antecedente por revisar, no una política validada para este objetivo. [R3]

Registrar desired, current y ready, retraso de reacción, Pods Pending y capacidad libre. El HPA no crea CPU física ni réplicas PostgreSQL. Mantener mínimo y margen para la ráfaga y el fallo de un nodo. VPA en recomendación; autoescalado de nodos, Kafka, KEDA y Service Mesh se evalúan después como tratamientos separados, evitando atribuirles resultados de otra condición.

Etapa L05 de carga y significado de los usuarios

G6 construirá scripts k6 parametrizados en pruebas/ha-50000, con cuentas sintéticas, datos exclusivos, límites, salida estructurada y partición de trabajo entre generadores. No crear 50 000 credenciales dentro del Secret actual sin estudiar tamaño y costo: la autenticación presente recorre el mapa de claves. Un conjunto acotado de propietarios simula sesiones lógicas pero no demuestra 50 000 identidades independientes. Si se cambia autenticación, crear una etapa explícita y otra referencia.

Escenario	Definición reproducible	Interpretación
P piloto	100 y luego 1000 VU con pausas y duración fijadas	Calibración sin promesa de capacidad

Escenario	Definición reproducible	Interpretación
V volumen	50 000 intenciones nuevas en 600 s	Promedio 83.33 intenciones por segundo, no simultaneidad
U concurrencia	Rampa 1000, 5000, 10000, 25000 y 50000 VU activos	Sostener cada nivel cinco minutos si pasa el anterior
R ráfaga	50 000 intenciones repartidas en una ventana propuesta de 10 s	Tasa objetivo 5000 por segundo, prueba separada
I inventario	Un evento con 1000 cupos y demanda superior	Rechazo correcto y cero sobreventa

Las duraciones y escalones son propuestas que G6 debe congelar antes de medir. Para U cada VU consulta disponibilidad, intenta una reserva de una unidad con clave nueva, consulta su comprobante y espera una pausa definida. Separar catálogo inicial, calentamiento y retries del conteo de intenciones. Repetir iteraciones requiere inventario proporcional o un escenario explícito de agotamiento. No afirmar usuarios concurrentes a partir del número acumulado de claves.

En V y R usar modelo de llegada abierto con tasa y duración fijadas; en U usar VU activos y pausas fijadas. Un ensayo cerrado puede reducir solicitudes cuando la aplicación se ralentiza; por eso reportar también la tasa realizada. Sumar VU y tasas de generadores sin duplicar claves; UUID o prefijos de run_id y shard evitan colisiones. Validar CPU, RAM, red, archivos abiertos, conexiones y dropped_iterations del generador. Registrar incumplimiento de carga como resultado inválido o incompleto, no como aprobación del servidor. [R6]

Etapa L06 de fallos y disponibilidad

Ejecutar primero sin fallos. Luego, sobre una carga sostenible comprobada, ensayar por separado pérdida de un Pod API, mantenimiento mediante drain, caída abrupta de un trabajador, conmutación PostgreSQL y pérdida de una instancia de entrada. Una eliminación de Pod no equivale a perder el anfitrión. Un PDB limita ciertas expulsiones voluntarias; no impide fallos físicos ni hace segura cualquier eliminación directa. [R4]

G3 preparará para cada fallo un guion con recurso exacto, propietario, inicio y duración, alcance autorizado, observador externo, procedimiento de restauración y condición de aborto. Registrar t0 del fallo, primer error, último error y momento de estabilización, además de réplicas sanas y recuperación. No publicar comandos de apagar nodos hasta identificar el proveedor y laboratorio; el agente deberá generar scripts específicos revisables con vista previa. Nunca interrumpir contenedores Compose de otros sprints como sustituto.

Proponer p95 de creación menor o igual a 800 ms y p99 menor o igual a 2000 ms en carga estable, y al menos 99.9 por ciento de respuestas elegibles correctas dentro del tiempo máximo congelado. El denominador de disponibilidad incluye intentos externos programados que no lograron

respuesta; registrar por separado validaciones, sold_out y conflictos esperados. Además medir intervalos temporales con un sondeo externo de reserva sobre datos dedicados y con capacidad suficiente. Una página de salud accesible no acredita que se pueda reservar.

Proponer recuperación del servicio dentro de 30 s para un fallo de Pod API y dentro de 120 s para nodo o promoción PostgreSQL. Son límites de evaluación, no resultados ni valores garantizados por Kubernetes. Un fallo con recuperación en 120 s puede incumplir 99.9 por ciento en una ventana corta; reportar ambos indicadores sin ocultarlo. Cero pérdida de confirmadas y cero sobreventa se evalúan por auditoría aun si hay errores transitorios.

Abortar inmediatamente por integridad rota, impacto externo o pérdida de control del ensayo. Como propuesta operativa, detener la escalada si hay OOM, disco bajo el 20 por ciento libre, o más del 5 por ciento de errores técnicos durante 60 s; marcar el nivel como no alcanzado. Una prueba con carga no enviada o captura perdida se etiqueta incompleta y se repite solo después de corregir su causa.

Etapa L07 de análisis y demostración

G6 coordinará al menos tres repeticiones por condición y nivel aprobado, con estado inicial equivalente y orden bloqueado o aleatorizado. Guardar distribución y percentiles por corrida, diferencias y límites de incertidumbre; tres repeticiones son exploratorias, no prueba automática de equivalencia. Comparar A con B para replicación y B con C para adaptación de capacidad, informando recursos realmente usados. La comparación no demuestra superioridad de microservicios sobre un monolito que no existe.

El registro de ensayo incluirá run_id, Git SHA y árbol sucio, versiones, perfil y dominios de fallo, manifiestos y digest, datos iniciales, propietarios sintéticos, semilla, tratamiento, VU objetivo y efectivo, tasas, pausas, duración, fallo, serie del cliente, métricas, auditorías y veredicto. Guardar en evidencias/ha-50000 por run_id; mantener secretos fuera. Exportar una tabla CSV de corridas y un informe Markdown y Word con rutas de evidencias. Declarar por separado planificado, ejecutado, aprobado, fallido, incompleto o pendiente.

Preparar una demostración de 25 minutos: arquitectura y dominios de fallo durante cinco; reserva funcional durante cinco; carga y réplicas durante cinco; fallo previamente ensayado y recuperación durante cinco; auditoría y límites durante cinco. La prueba máxima se ejecuta antes y sus resultados se muestran con fecha. En vivo usar un nivel seguro y una base distinta. Si 50 000 no se alcanzan, mostrar la frontera medida y su causa. No convertir un video histórico en evidencia de ejecución en vivo.

Reparto y orden de los prompts

Responsable	Construcción	Dependencias y aceptación
Coordinación L00	Inventario y ruta concreta	Acceso real y plan de recursos
G1 L01 y L02	Plataforma y despliegue	L00; entrada y réplica comprobadas

Responsable	Construcción	Dependencias y aceptación
G4 L03	Datos HA e integridad	L00 y plataforma; auditoría y recuperación
G5 L04	Instrumentación y tableros	Aplicación y datos; señales contrastadas
G2 L04	Políticas y tratamientos	Métricas disponibles y presupuesto
G3 L06	Seguridad y fallos	Plataforma estable y guiones ensayados
G6 L05 y L07	Generación, experimento e integración	Contratos de todos los grupos

Los prompts están en documentacion/fase-02/prompts/laboratorio-ha-50000. Dar a un agente nuevo primero 00-prompt-maestro.md. Para una tarea específica entregar también 01-contrato-comun.md y el prompt del grupo correspondiente. Son archivos de instrucciones para futuros agentes; esta entrega no crea agentes ni implementa el clúster.

Fuentes y comprobación

[R1] Kubernetes. Creating Highly Available Clusters with kubeadm. https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/

[R2] Kubernetes. Pod Topology Spread Constraints. https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/

[R3] Kubernetes. Horizontal Pod Autoscaling. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[R4] Kubernetes. Disruptions. https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

[R5] PostgreSQL. Standby servers and synchronous replication. https://www.postgresql.org/docs/current/warm-standby.html

[R6] Grafana k6. Arrival rate VU allocation. https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/arrival-rate-vu-allocation/

[R7] Prometheus. Histograms and summaries. https://prometheus.io/docs/practices/histograms/

[R8] CloudNativePG. Failover. https://cloudnative-pg.io/docs/current/failover/ La selección del operador y su versión queda pendiente de compatibilidad con el clúster elegido.

Fuentes locales inspeccionadas: laboratorio/scripts, laboratorio/configuracion, aplicacion/reservas/api.py, store.py, config.py, schema.sql y manage.py; frontend/deploy/base; DOC-01-02 y registros de estado. Consulta técnica del 20 de octubre de 2026. Esta guía no acredita ejecución del laboratorio ni capacidad para 50 000 usuarios.